

TECHNICAL UNIVERSITY OF MOLDOVA

Faculty of Computers, Informatics and Microelectronics

Software Engineering

OPERATING SYSTEMS

Laboratory Work No. 12

Producer, Consumer and the Other Tools

Performed by:
Bianca-Andreea Tataroi
Group FAF-242

Checked by:
Patricia Reițman

Chisinau, 2026

Contents

1 Introduction **2**

2 Objectives **2**

3 Part 1 - Producer and Consumer **2**
3.1 Implementation 2
3.2 Successful Execution 2
3.3 Incorrect Lock Ordering Experiment 3

4 Part 2 - The Condition Variable **4**
4.1 Implementation 4
4.2 Execution 4

5 Part 3 - Readers and Writers **4**
5.1 Implementation 4
5.2 Execution 5

6 Part 4 - Build a Deadlock, Then Remove It **5**
6.1 Original Deadlock 5
6.2 Ten Runs Without Delays 5
6.3 Fixed Version 6

7 Answers to Observation Questions **7**
7.1 Part 1 - Producer and Consumer 7
7.2 Part 2 - Condition Variable 7
7.3 Part 3 - Readers and Writers 7
7.4 Part 4 - Deadlock 8

8 Conclusion **8**

9 GitHub Repository **8**

1 Introduction

Synchronization is essential when multiple threads share memory and resources. Without coordination, concurrent programs may produce race conditions, access incomplete data, or become permanently blocked. In this laboratory, Linux POSIX threads, semaphores, mutexes, and condition variables were used to implement four classic synchronization examples: producer-consumer, ordered communication, readers-writers, and deadlock prevention.

2 Objectives

The objectives of this laboratory work were:

- Implement a bounded producer-consumer buffer using semaphores and a mutex.
- Coordinate two threads with a condition variable.
- Allow concurrent readers while ensuring exclusive access for writers.
- Reproduce deadlock, investigate its dependence on scheduling, and remove circular waiting.
- Compile and test all programs on Linux using `gcc -Wall -pthread`.

3 Part 1 - Producer and Consumer

3.1 Implementation

The program `prodcons.c` uses a circular buffer with `SIZE = 4` and processes eight items. The semaphore `empty` starts at 4 and counts free slots, while `full` starts at 0 and counts occupied slots. The mutex `lock` protects the buffer indices and shared item count.

```
sem_wait(&empty);
pthread_mutex_lock(&lock);
/* insert item into circular buffer */
pthread_mutex_unlock(&lock);
sem_post(&full);
```

The consumer performs the complementary operations, waiting for `full` before removing an item and posting `empty` after removal.

3.2 Successful Execution

The program was compiled and executed with:

```
gcc -Wall -pthread -o prodcons prodcons.c
./prodcons
```

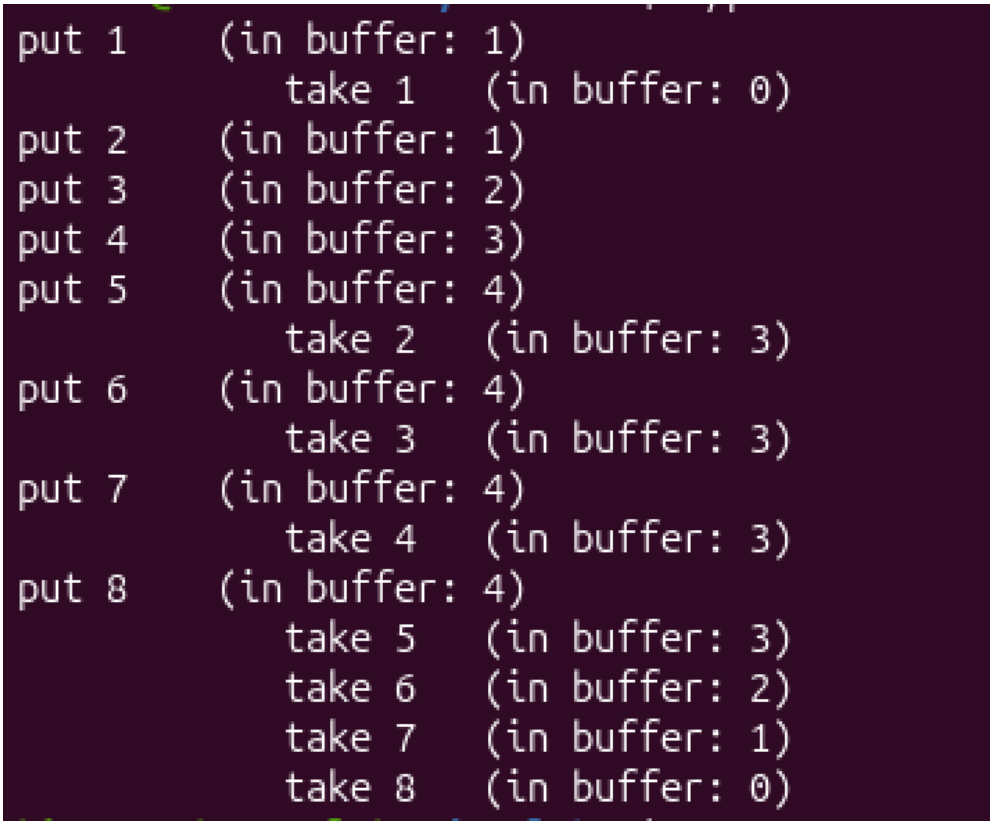


Figure 1: Successful producer-consumer execution with buffer occupancy between 0 and 4

The observed output shows all eight items being inserted and then removed. The consumer sometimes runs immediately after the first insertion; this is normal because thread scheduling is nondeterministic. The buffer finishes empty.

3.3 Incorrect Lock Ordering Experiment

The first two synchronization operations in `producer()` were reversed:

```
pthread_mutex_lock(&lock);
sem_wait(&empty);
```

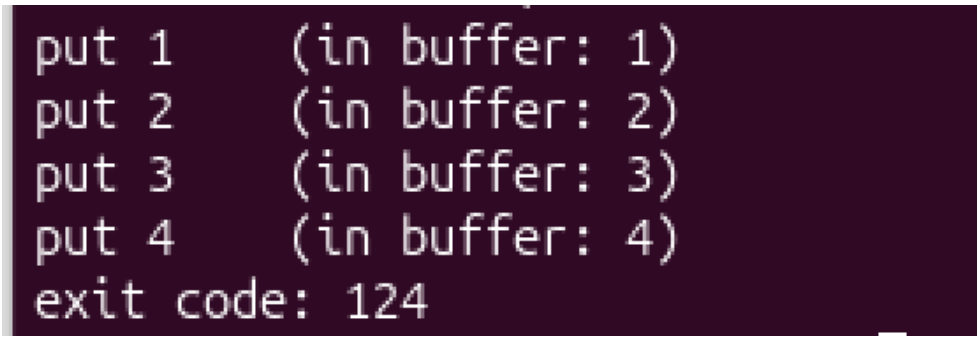


Figure 2: Producer-consumer deadlock after filling the buffer; exit code 124

The output stopped with four items in the buffer. The producer held the mutex and waited for a free slot, preventing the consumer from acquiring the mutex to remove an

item. The `timeout` utility terminated the deadlocked program with exit code `124`. The original safe order was then restored.

4 Part 2 - The Condition Variable

4.1 Implementation

In `condvar.c`, thread B (reader) must not access the shared message until thread A (writer) has initialized it. The mutex protects `message` and the shared state variable `ready`. The reader uses `pthread_cond_wait()`, which releases the mutex while waiting and reacquires it before returning.

```
while (!ready) {
    pthread_cond_wait(&changed, &lock);
}
```

4.2 Execution

```
gcc -Wall -pthread -o condvar condvar.c
./condvar
```

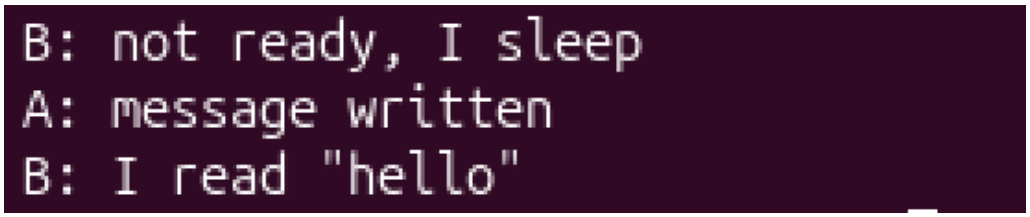


Figure 3: The reader waits until the writer publishes the message

The reader first reported that the message was not ready, the writer published `hello`, and the reader then printed the same message. Replacing `while` with `if` may still work in this simple execution, but `while` is required for robust correctness because of spurious wakeups or a condition changing before the awakened thread proceeds.

5 Part 3 - Readers and Writers

5.1 Implementation

In `readers.c`, three reader threads and two writer threads share an integer. The semaphore `rlock` protects updates to the reader count, and `room` ensures that writers access the shared value exclusively. The first reader enters by acquiring `room`; the last reader releases it.

```
if (++readers == 1)
    sem_wait(&room);
/* readers may access the value concurrently */
```

```
if (--readers == 0)
    sem_post(&room);
```

5.2 Execution

```
gcc -Wall -pthread -o readers readers.c
./readers
```

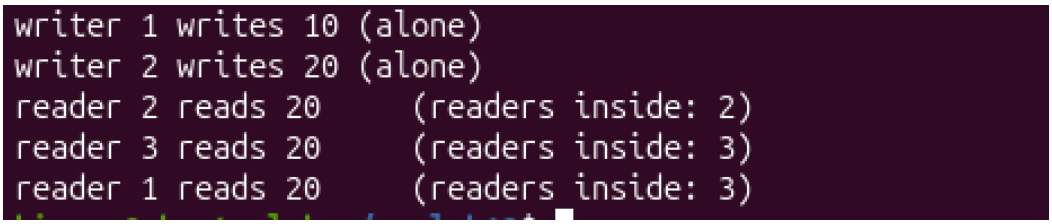


Figure 4: Readers-writers output: both writers finish and three readers share the room

In the observed run, writer 1 updated the value to 10 and writer 2 updated it to 20 before the readers printed their observations. This ordering differs from the example in the lab handout but is valid: the scheduling order can vary. The readers all displayed the value 20 and the number of concurrent readers reached 3.

6 Part 4 - Build a Deadlock, Then Remove It

6.1 Original Deadlock

The original `deadlock.c` creates two threads that acquire mutexes A and B in opposite orders. Thread one holds A and waits for B, while thread two holds B and waits for A.

```
timeout 5 ./deadlock; echo "exit code: $?"
```

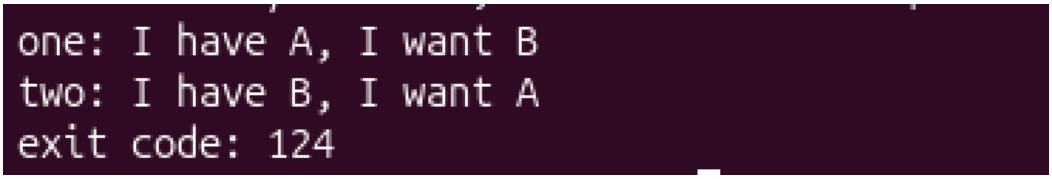


Figure 5: Circular wait between two threads; timeout exit code 124

The program printed both initial messages and then stopped progressing. Exit code 124 confirms that the timeout was reached.

6.2 Ten Runs Without Delays

Both `sleep(1)` statements were temporarily removed and the original deadlock-prone program was executed ten times using `timeout`. The recorded runs included both normal

completion (exit code 0) and deadlock (exit code 124). The screenshot includes runs 6–10, where runs 6 and 9 deadlocked, while runs 7, 8 and 10 completed.

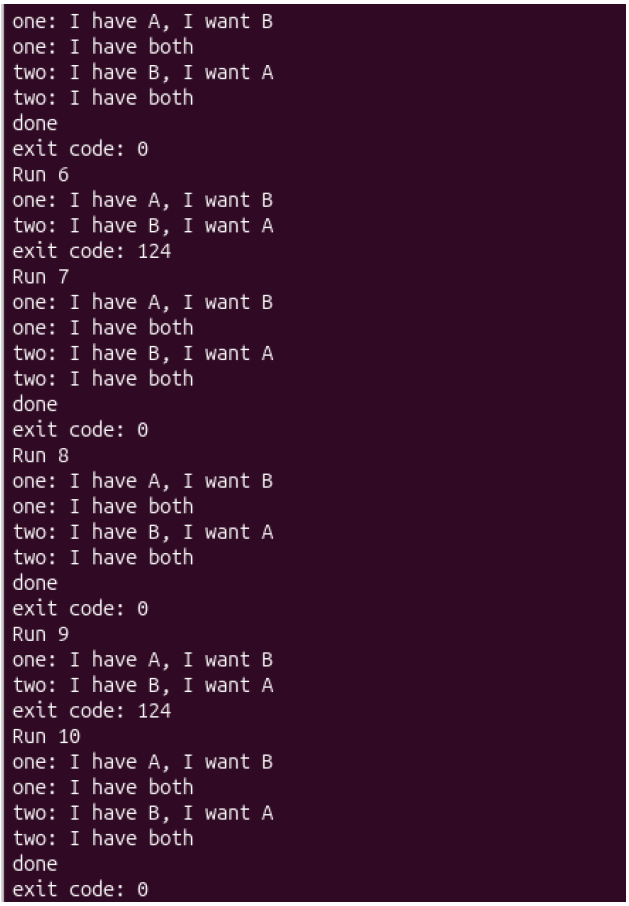


Figure 6: Repeated executions show intermittent deadlock after removing the delays

This shows that removing the delays does not solve the synchronization problem. It merely changes the timing, making the defect nondeterministic and harder to reproduce.

6.3 Fixed Version

The second thread was changed to acquire the locks in the same order as the first thread: first A, then B. The unlock operations were also ordered consistently in reverse order.

```
pthread_mutex_lock(&A);
pthread_mutex_lock(&B);
/* critical section */
pthread_mutex_unlock(&B);
pthread_mutex_unlock(&A);
```

```
one: I have A, I want B
one: I have both
two: I have A, I want B
two: I have both
done
exit code: 0
```

Figure 7: Successful execution after consistent lock ordering; exit code 0

Both threads were able to acquire the locks, finish their critical sections, and print `done`. The process exited with code 0.

7 Answers to Observation Questions

7.1 Part 1 - Producer and Consumer

1. **Why does the buffer never exceed four items?** `sem_wait(&empty)` blocks the producer when no free slots remain. The consumer signals a free slot using `sem_post(&empty)`.

2. **Who waits at the beginning, and why?** The consumer waits because the `full` semaphore is initialized to zero. The waiting call is `sem_wait(&full)`.

3. **What happens when the mutex is acquired before waiting on empty?** Once the buffer is full, the producer waits for an empty slot while holding the mutex. The consumer cannot acquire that mutex to remove an item, causing deadlock.

7.2 Part 2 - Condition Variable

1. **Which variable records whether the message is ready?** The shared variable `ready`, which the writer changes from 0 to 1.

2. **Does replacing while with if still work?** It works in a typical run of this small example, but it does not provide the same correctness guarantee.

3. **Why is while necessary?** A thread may experience a spurious wakeup or awaken when another thread has changed the condition. Rechecking `ready` avoids proceeding when it remains false.

7.3 Part 3 - Readers and Writers

1. **Which two lines permit multiple readers but exclude writers?** The first reader acquires `room` using `if (++readers == 1) sem_wait(&room);`; the last reader releases it using `if (--readers == 0) sem_post(&room);`.

2. **Why is rlock needed?** It protects the shared `readers` counter from concurrent increments and decrements. Without it, race conditions could cause an incorrect count or improper room locking.

3. **What happens if readers keep arriving?** A waiting writer may be postponed indefinitely because the room may never become empty. This is writer starvation.

7.4 Part 4 - Deadlock

1. **Who holds what, and what is the circular wait?** Thread one holds A and waits for B; thread two holds B and waits for A. The cycle is `one -> B -> two -> A -> one`.

2. **Does removing sleep always prevent hanging?** No. In the displayed repeated tests, some runs terminated and others timed out (including runs 6 and 9). The bug becomes intermittent because the schedule determines whether both threads acquire their first lock before requesting the second.

3. **Why does consistent lock ordering prevent this deadlock?** Acquiring A before B in both threads removes the circular-wait condition, so this pair of locks cannot deadlock in that way.

8 Conclusion

In this laboratory work, four synchronization problems were implemented and tested in Linux. The producer-consumer program demonstrated how semaphores limit a shared buffer and how incorrect lock ordering can block progress. The condition-variable example showed reliable notification between a writer and a reader. The readers-writers program allowed concurrent readers while ensuring that writers accessed the shared value exclusively. Finally, the deadlock tests showed that timing can hide concurrency defects, while consistently acquiring resources in the same order removes circular waiting. All four programs produced the expected type of behavior in the provided screenshots.

9 GitHub Repository

The required submission contains `prodcons.c`, `condvar.c`, `readers.c`, `deadlock.c` (fixed), and `report.md`. The intended course repository is:

https://github.com/Biancatataroi/Operating-Systems-Lab12